Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия
Дисциплина «*Сервис-ориентированная архитектура*»

Отчёт
По лабораторной работе №2

Вариант: 21

Сдающие:
Петров Вячеслав Маркович
Елисеев Константин Иванович

группа P3408

Преподаватель:
Кривоносов Егор Дмитриевич

г. Санкт-Петербург, 2026 г.

Задание

На основе разработанной в рамках лабораторной работы #1 спецификации реализовать два веб-сервиса и использующее их API клиентское приложение.

Требования к реализации и развёртыванию сервисов:

- Первый ("вызываемый") веб-сервис должен быть реализован на фреймворке Spring MVC REST и развёрнут в окружении под управлением сервера приложений Tomcat.
- Второй веб-сервис должен быть реализован на фреймворке Spring MVC REST, развёрнут в окружении под управлением ещё одного экземпляра сервера приложений Tomcat и вызывать REST API первого сервиса.
- **Для обоих сервисов необходимо реализовать все функции, задокументированные в API, в строгом соответствии со спецификацией!**
- Доступ к обоим сервисам должен быть реализован по протоколу https с самоподписанным сертификатом сервера. Доступ к сервисам посредством http без шифрования должен быть запрещён.

Требования к клиентскому приложению:

- Клиентское приложение может быть написано на любом веб-фреймворке, который можно запустить на сервере helios.
- Приложение должно обеспечить полный набор возможностей, предоставляемых API обоих сервисов - включая сортировку, фильтрацию и постраничный вывод элементов коллекции.
- Приложение должно преобразовывать передаваемые сервисами данные в человеко-читаемый вид - параграф текста, таблицу и т.д.
- Клиентское приложение должно информировать пользователя об ошибках, возникающих на стороне сервисов, в частности, о том, что сервису были отправлены невалидные данные.

Оба веб-сервиса и клиентское приложение должны быть развёрнуты на сервере helios.

Выполнение

Исходный код

https://github.com/virdxk/ticket-api-openapi

- https://github.com/virdxk/ticket-api-openapi/tree/main/ticket-service - Ticket Service
- https://github.com/virdxk/ticket-api-openapi/tree/main/person-service - Person Service
- https://github.com/virdxk/ticket-api-openapi/tree/main/booking-service - Booking Service
- https://github.com/virdxk/ticket-api-openapi/tree/main/common - общий модуль: обработка ошибок, разбор фильтров и сортировки, поддержка JSON Merge Patch
- https://github.com/virdxk/ticket-api-openapi/tree/main/client - клиентское приложение
- https://github.com/virdxk/ticket-api-openapi/tree/main/deploy - скрипты развёртывания на helios

Реализация сервисов

Ticket Service, Person Service и Booking Service написаны на Java 17 и Spring Boot 3.5.16 со Spring MVC REST. Maven собирает каждый сервис в WAR-архив. Интерфейсы контроллеров, модели и клиенты Booking Service генерируются из исходных YAML-спецификаций плагином openapi-generator-maven-plugin. Сгенерированный код вручную не изменяется.

Развёртывание

Каждый сервис развёрнут в отдельном экземпляре Tomcat 10.1.60 на helios.

Публичные запросы идут через se.ifmo.ru. Для каждого сервиса в public_html есть каталог с .htaccess и PHP-прокси, который пересылает запрос в Tomcat по HTTPS и возвращает ответ. Ticket Service использует порт 41390, Person Service порт 41391, Booking Service порт 41392. Клиент лежит в public_html/lab2. Настройка ddl-auto=validate проверяет существующую схему PostgreSQL при запуске и не изменяет её.

Компонент	Публичный адрес
Ticket Service	https://se.ifmo.ru/~s412939/ticket-service/tickets

Person Service	https://se.ifmo.ru/~s412939/person-service/persons
Booking Service	https://se.ifmo.ru/~s412939/booking-service/booking/bookings
Клиент	https://se.ifmo.ru/~s412939/lab2/
Swagger UI	https://se.ifmo.ru/~s412939/

Доступ по https

Самоподписанный сертификат создаёт утилитой keytool скрипт развёртывания при первом запуске на helios: хранилище ключа keystore.p12, хранилище доверенных сертификатов truststore.p12 и сертификат server.crt. Пути к ним и пароли указаны в server.xml Tomcat и в application.properties Booking Service. В server.xml есть только https-коннектор, поэтому запросы по http отклоняются.

```
$ curl http://localhost:41390/tickets
Bad Request
This combination of host and port requires TLS.


$ curl https://localhost:41390/tickets
curl: (60) SSL certificate problem: self-signed certificate


$ curl --cacert ~/public_html/ticket-service/server.crt
https://localhost:41390/tickets/2
{"id":2,"name":"2222","coordinates":{"x":1.0,"y":2.0},"creationDate":"2026-10-
09T19:30:31.184Z","price":null,"comment":"","type":null,"venue":null}
```

Клиентское приложение

https://se.ifmo.ru/~s412939/lab2/

Клиент написан на React и собирается Vite. Для каждой коллекции есть список с фильтрацией, многоуровневой сортировкой и постраничным выводом, получение, создание, частичное обновление и удаление элемента.

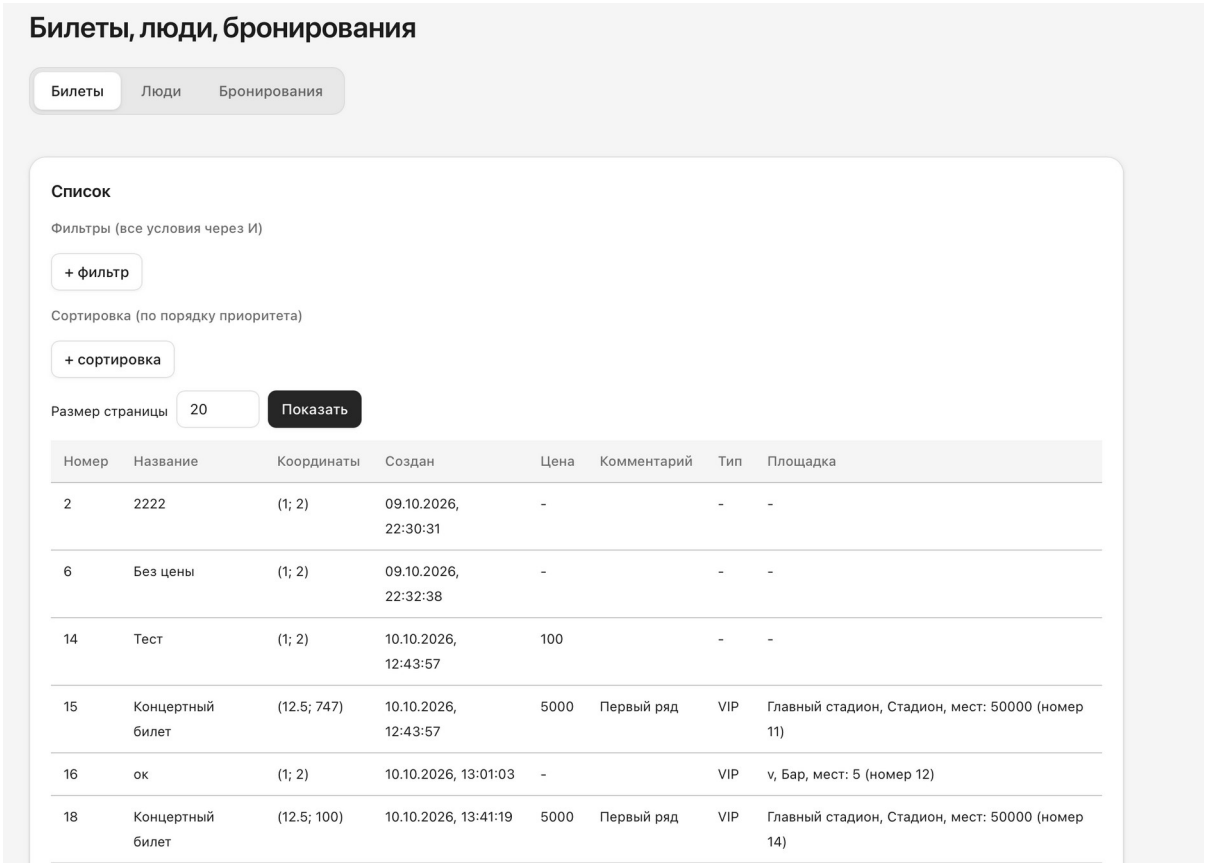


Рис 1. Список билетов

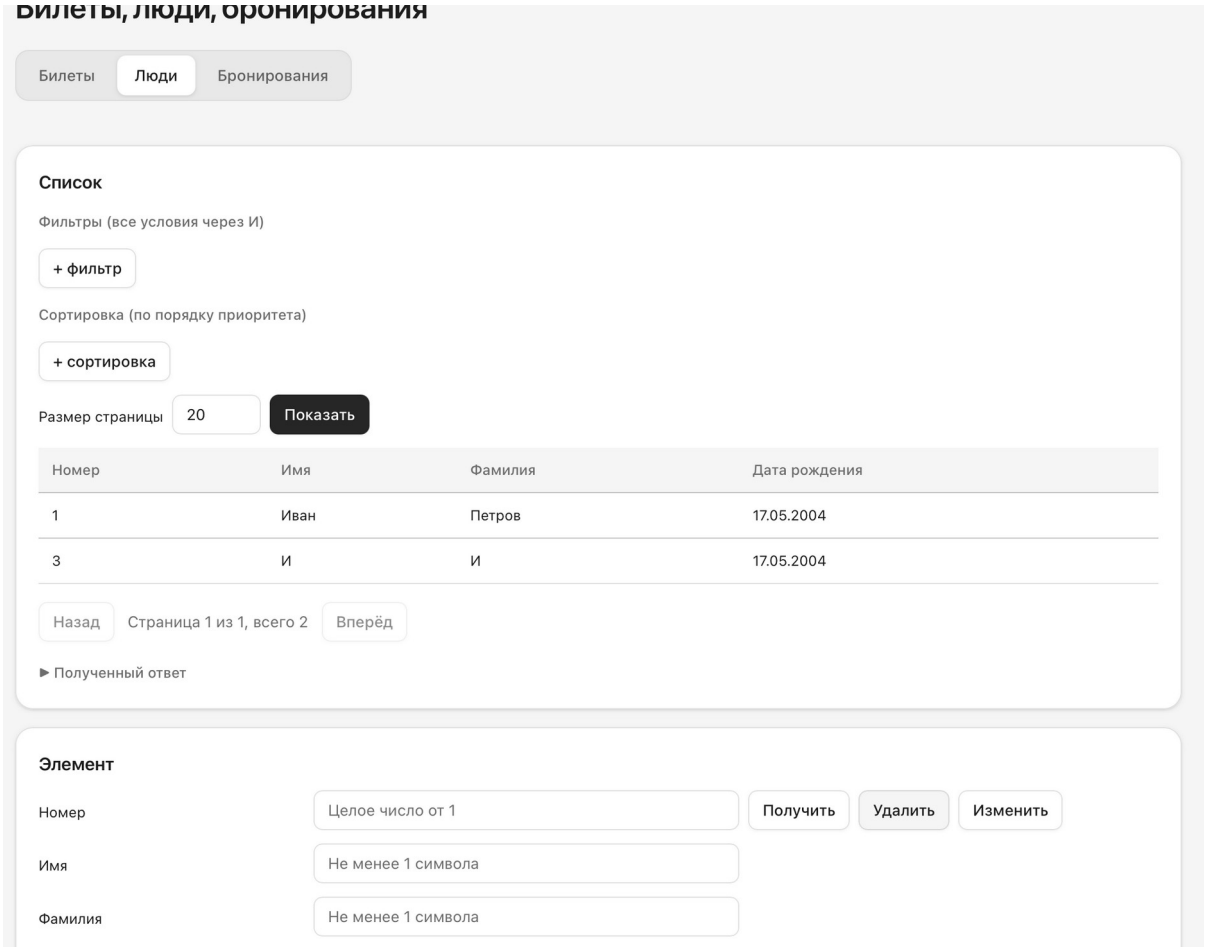


Рис 2. Список людей

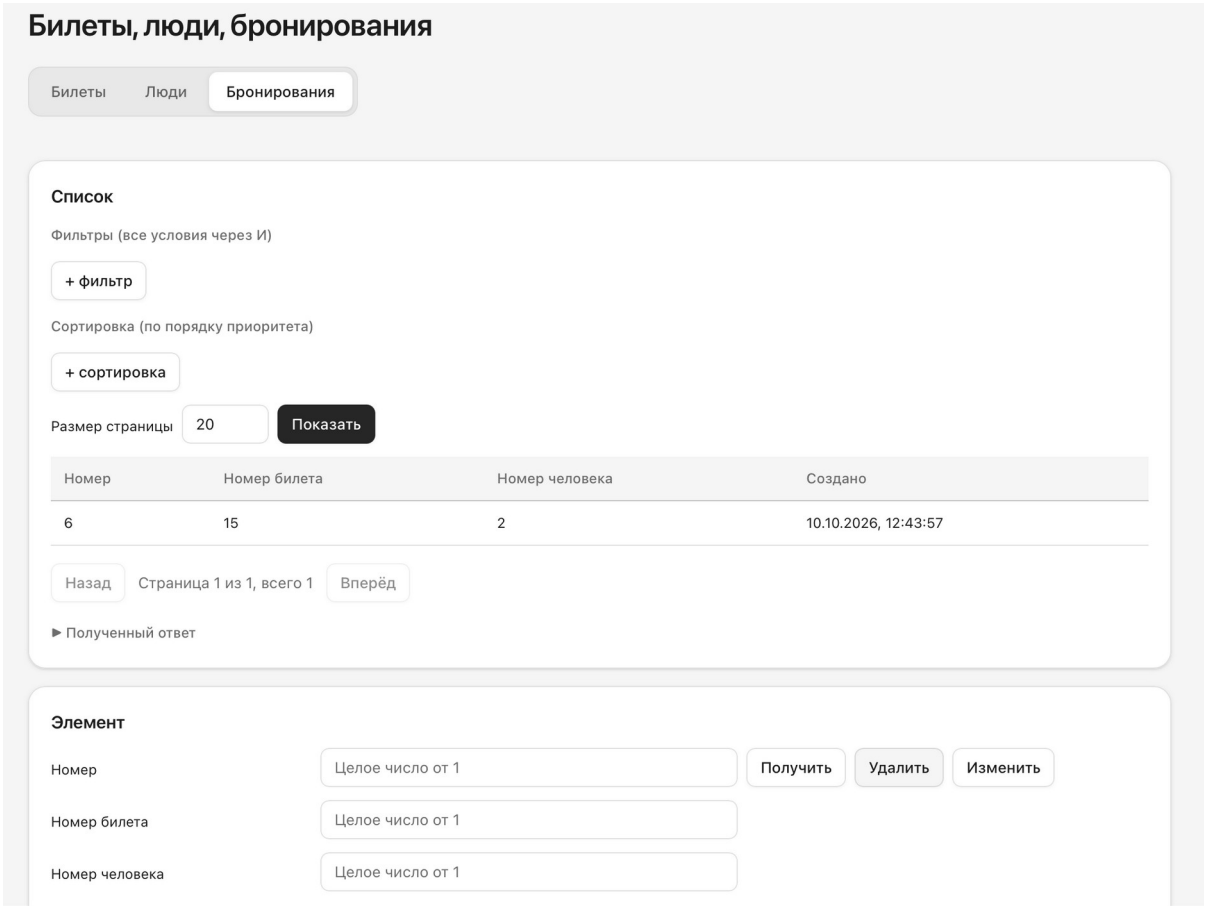


Рис 3. Список бронирований

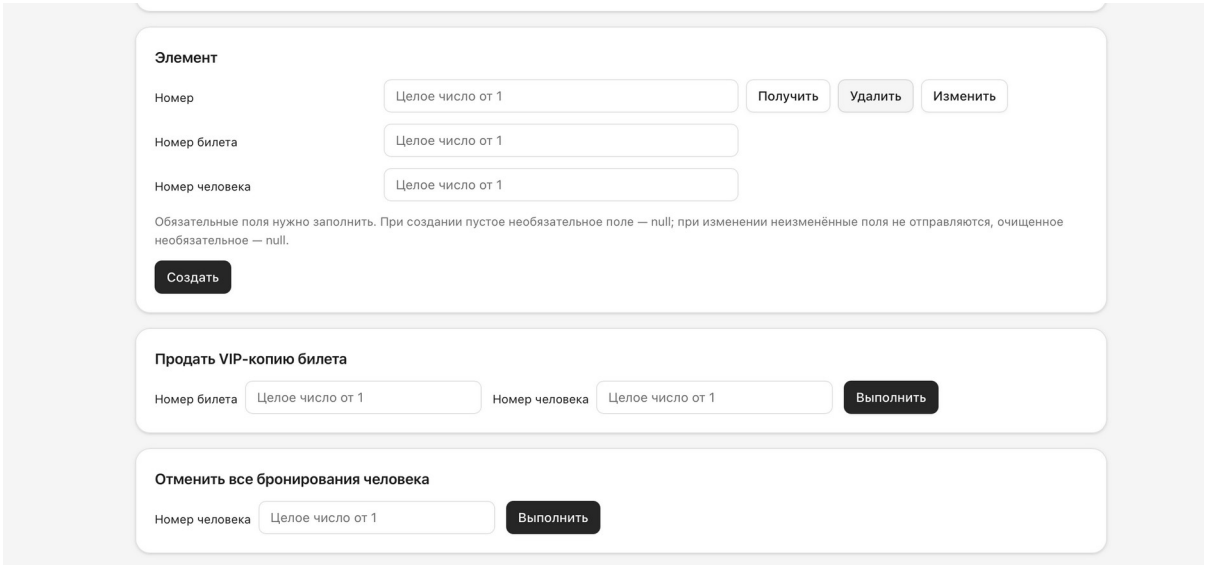


Рис 4. Создание бронирования и специальные операции

Swagger UI

https://se.ifmo.ru/~s412939/

В спецификациях указаны публичные https-адреса сервисов, поэтому операции можно выполнять прямо из Swagger UI.

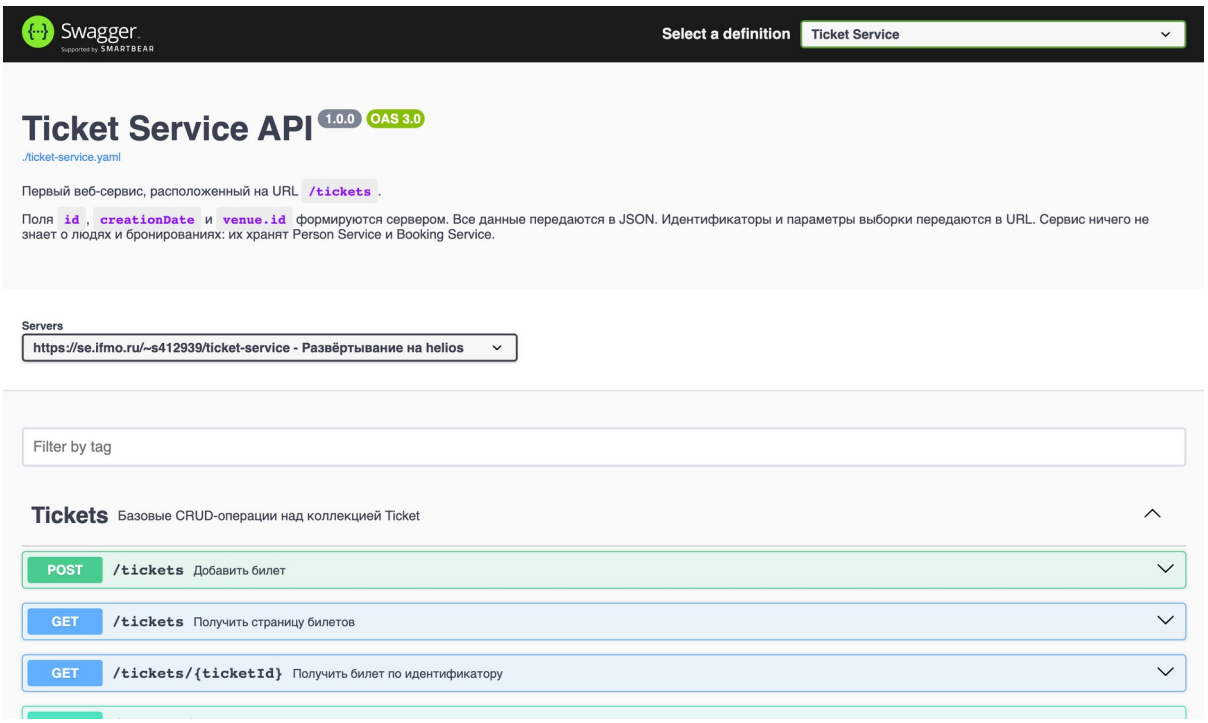


Рис 5. Swagger UI для Ticket Service

Выводы

Реализованы три сервиса на Spring MVC REST и клиент на React. Генерация из OpenAPI связывает контроллеры, модели и REST-клиенты с исходными контрактами. Запросы к PostgreSQL выполняют фильтрацию, сортировку и постраничную выборку. Ошибки имеют единый формат и русские и английские переводы. JSON Merge Patch сохраняет различие между отсутствующим полем и явным null. Для развёртывания подготовлены WAR-архивы, отдельные экземпляры Tomcat и HTTPS с самоподписанным сертификатом.